

Q1. What is a thread in Java? How does it differ from a process?

Thread:

A thread is the smallest unit of execution within a program. It represents a single sequential flow of control that can run independently, while sharing the same memory space (heap) as other threads in the same program. In Java, a thread is represented by the Thread class or the Runnable interface.

Process:

A process is an independent, self-contained running instance of a program, with its own separate memory space, allocated by the operating system. A single process can contain one or more threads.

Key differences:

- Memory: threads within the same process share the same memory space, while separate processes each have their own independent memory space.
- Creation cost: creating a new thread is generally lighter and faster than creating a new process, since threads share existing resources.
- Communication: threads can communicate directly by sharing variables in memory; processes generally require inter-process communication mechanisms (such as sockets or pipes) provided by the operating system.
- Isolation: a crash in one process typically does not affect other processes, while an unhandled error in one thread can potentially affect the entire process it belongs to.

Q2. Identify concurrency issues in shared memory.

When multiple threads access shared memory concurrently, several classic problems can occur:

- Race condition: two or more threads access and modify shared data at the same time, and the final result depends unpredictably on the exact timing/order of execution.
- Data inconsistency: without proper synchronization, one thread may read a shared variable while another thread is in the middle of updating it, leading to incorrect or partially updated values.
- Deadlock: two or more threads wait indefinitely for each other to release resources (such as locks) that they each need, so none of them can proceed.
- Starvation: a thread is perpetually denied access to a needed shared resource because other threads are repeatedly given priority.

- Visibility issues: a change made to a shared variable by one thread may not be immediately visible to other threads due to caching, unless proper synchronization or the volatile keyword is used.

For example, if two threads both execute `count++` on a shared, unsynchronized variable `count`, some increments can be lost because the increment operation is not atomic -- it involves separate read, modify, and write steps that can interleave between threads.

Q3. Define thread lifecycle in Java with example.

A Java thread moves through several distinct states during its lifecycle:

- New: the thread object has been created (using `new Thread(...)`) but `start()` has not yet been called.
- Runnable: after `start()` is called, the thread becomes eligible to run and is waiting for the CPU scheduler to allocate processing time to it.
- Running: the thread is actively executing its task, having been selected by the scheduler.
- Blocked / Waiting: the thread is temporarily inactive, either waiting for a resource, a lock, or another thread's action (for example, due to `sleep`, `wait`, or `join`).
- Terminated: the thread has completed its `run()` method, or has otherwise stopped executing, and cannot be restarted.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

public class LifecycleDemo {
    public static void main(String[] args) {
        MyThread t = new MyThread(); // New state
        t.start();                    // moves to Runnable, then Running
    }
}
```

In this example, `t` is in the New state right after creation, moves to the Runnable state once `start()` is called, is placed in the Running state when the scheduler executes its `run()` method, and finally moves to the Terminated state once `run()` finishes.